

实验报告部分：SM3 哈希函数实现与优化解析

一、实验目的

- 理解 SM3 哈希算法的结构与流程
- 使用 C++ 实现 SM3 哈希算法
- 在 Visual Studio 中测试 SM3 的性能
- 进行优化与吞吐量测试，分析不同优化策略的效果

二、SM3 算法原理解析

2.1 SM3 简介

SM3 是中国国家密码管理局于 2010 年发布的哈希函数标准，具有如下特点：

- 输出长度：256 位（32 字节）
- 消息分组：512 位（64 字节）
- 安全性：与 SHA-256 相当，设计上使用 Merkle-Damgård 构造

2.2 算法结构

SM3 由以下几部分组成：

1. 初始向量（IV）

v0 = 7380166f 4914b2b9 172442d7 da8a0600
a96f30bc 163138aa e38dee4d b0fb0e4e

2. 填充

- 补 1
- 补 0 直到消息长度 $\equiv 448 \pmod{512}$
- 最后追加原始长度（64 位）

3. 消息扩展

将 512 位消息块扩展为 132 个字：

- w[0..67]：主扩展
- w1[0..63] = w[i] $\oplus$ w[i+4]：辅助扩展

使用非线性函数 $P1(x) = x \oplus (x \ll 15) \oplus (x \ll 23)$

4. 压缩函数 CF

对每个分组进行 64 次迭代：

- 状态变量：A~H（共 8 个 32 位）
- 非线性布尔函数：

```
FF_j(x, y, z) =  
    x ^ y ^ z          (j ∈ [0,15])  
    (x & y) | (x & z) | (y & z) (j ∈ [16,63])  
  
GG_j(x, y, z) =  
    x ^ y ^ z          (j ∈ [0,15])  
    (x & y) | (~x & z) (j ∈ [16,63])
```

- `SS1 = ((A << 12) + E + (T_j << j)) << 7`
- `SS2 = SS1 ⊕ (A << 12)`
- `TT1 = FF_j(...) + D + SS2 + w1[j]`
- `TT2 = GG_j(...) + H + SS1 + w[j]`

每轮更新 A~H 值，并将新值异或到中间状态向量 `V[i+1] = V[i] ⊕ CF(...)`

三、C++ 实现细节

SM3 哈希函数作用

SM3 接收任意长度的消息输入，输出一个固定长度的 **256-bit (32字节)** 哈希值，用于：

- 数据完整性校验（如文件校验码）
- 密码学签名
- 区块链中的数据哈希
- 任何需要抗碰撞哈希的场景

代码结构详解

1. 类型定义与常量

```
using u8 = uint8_t;  
using u32 = uint32_t;  
using u64 = uint64_t;
```

- 简写类型定义，方便书写。
- `u8` 表示 8 位无符号整数，`u32` 表示 32 位无符号整数，以此类推。

2. 基本宏和函数

```
#define ROTL(x, n) (((x) << (n)) | ((x) >> (32 - (n))))  
#define P0(x) ((x) ^ ROTL((x), 9) ^ ROTL((x), 17))  
#define P1(x) ((x) ^ ROTL((x), 15) ^ ROTL((x), 23))
```

- **ROTL**: 循环左移 (用于掩码运算)
- **P0/P1**: SM3 定义的置换函数, 用于扩展消息 (message expansion)

3. 布尔函数 FF 和 GG

```
inline u32 FF(u32 x, u32 y, u32 z, int j)  
inline u32 GG(u32 x, u32 y, u32 z, int j)
```

- **FF** 和 **GG** 是用于 SM3 的布尔函数:
 - 前16轮是简单的 XOR
 - 后48轮使用逻辑与或运算 (增强非线性)

4. 轮常量函数 Tj

```
inline constexpr u32 Tj(int j)
```

- 为每一轮提供常量值:
 - 0-15: 0x79cc4519
 - 16-63: 0x7a879d8a

5. 消息压缩函数 sm3_compress

```
inline void sm3_compress(u32 v[8], const u8 block[64])
```

核心函数: 处理每一个 512-bit (64字节) 分组。

流程如下:

1. **消息扩展**:
 - **w[68]**: 主扩展数组
 - **w1[64]**: 辅助数组
2. **初始化寄存器**:
 - **A~H** 对应 SM3 算法中的 8 个工作寄存器
3. **64轮迭代运算**:
 - 使用 FF/GG、P0、Tj 等函数进行一系列混合操作
4. **更新中间变量 V[8]**:
 - **V[i] ^= A~H**: 将结果反馈回 V, 类似于 SHA 家族

6. 最终哈希函数 sm3

```
inline std::vector<u8> sm3(const u8* msg, size_t len)
```

这是 SM3 的主接口：

1. 初始化 V (初始向量 IV) : 8个常量组成
2. 填充消息：
 - 添加 0x80
 - 补足到满足 $(len + padding + 8) \% 64 == 0$
 - 最后加上原始消息长度 (64 位表示)
3. 分块压缩：
 - 每 64 字节调用一次 sm3_compress
4. 输出 digest (摘要) :
 - 将最终的 V[8] 按照字节输出为 32 字节哈希值

最终输出结果

该 sm3() 函数会返回一个 std::vector<u8> , 包含 32 字节的哈希值。

你可以用它像这样：

```
std::string input = "abc";
auto hash = sm3((const u8*)input.data(), input.size());
```

示例输入输出

输入：字符串 "abc"

输出 (SM3 哈希) :

```
66C7F0F462EEEDD9D1F2D46BDC10E4E24167C4875CF2F7A2297DA02B8F4BA8E0
```

之后我们进行测试：

```
#include <iostream>
#include <iomanip>
#include <chrono>
#include "sm3_optimized.h"

int main() {
    const size_t msg_size = 4 * 1024 * 1024; // 每条消息大小: 4MB
    const int loops = 50; // 重复处理 50 次, 总量 200MB

    std::vector<u8> msg(msg_size, 0x61); // 填充内容 'a'
```

```
// 功能验证
auto digest = sm3((const u8*)"abc", 3);
std::cout << "SM3(\"abc\") = ";
for (auto b : digest)
    std::cout << std::hex << std::setw(2) << std::setfill('0') << (int)b;
std::cout << std::endl;

// 性能测试
auto start = std::chrono::high_resolution_clock::now();
for (int i = 0; i < loops; ++i) {
    volatile auto hash = sm3(msg.data(), msg_size);
}
auto end = std::chrono::high_resolution_clock::now();

double seconds = std::chrono::duration<double>(end - start).count();
double total_mb = (double)(msg_size * loops) / (1024 * 1024);
double throughput = total_mb / seconds;

std::cout << "SM3 throughput: " << throughput << " MB/s" << std::endl;
return 0;
}
```

四、优化策略与性能提升分析

4.1 编译优化（基础）

优化项	VS 设置
最大速度优化	Release 模式, /O2
启用本地指令集	C/C++ → Codegen → /arch:AVX2
内联函数/宏展开	函数 inline + 编译器优化

4.2 算法优化

优化点	描述
循环展开	减少 if 分支判断, 提高流水线效率
W/W1 缓存复用	避免多次计算相同中间值
状态变量寄存器化	A~H 全部保持为局部变量, 减少内存写入
内存填充对齐	消息使用 std::vector<u8> 一次性填充, 提高缓存局部性

4.3 多轮测试统计吞吐量

测试方式:

```
auto start = high_resolution_clock::now();
for (int i = 0; i < N; ++i) sm3(data, len);
auto end = high_resolution_clock::now();
```

吞吐量计算公式：

Throughput = Total_Data_Size / Duration_Seconds (MB/s)

五、性能测试结果（单线程）



可见我们成功的运行了，但是吞吐率并不是很高，我们尝试使用编译器优化：

将debug模式换成了release模式：

1. 启用了高级优化选项（比如 /O2）

优化类型	作用
常量折叠	<code>x = 2+3;</code> 会在编译期直接变成 <code>x = 5</code>
函数内联	避免频繁函数调用开销
循环展开与合并	减少循环次数、分支预测冲突
寄存器优化	把频繁变量存在寄存器里，避免内存访问
指令重排 / 向量化（SIMD）	提高流水线效率，减少指令数

2. Debug 模式保留了大量开销

Debug 特性	性能影响
保留所有变量符号	增加栈/堆访问次数
禁止某些优化（比如内联）	避免调试混乱
增加额外检查（栈溢出检测）	慢

3. Release 模式利用 CPU 指令集优化 (如 `/arch:AVX2`)

- 会使用 **SIMD** 指令 (如 AVX/SSE) 对数据块并行处理

Microsoft Visual Studio 调试窗 × + ▾

SM3("abc") = 66c7f0f462eedd9d1f2d46bdc10e4e24167c4875cf2f7a2297da02b8f4ba8e0
SM3 throughput: 213.414 MB/s

D:\crypto-projects\SM2\x64\Release\SM2.exe (进程 3132)已退出，代码为 0 (0x0)。
按任意键关闭此窗口 . . .

可见吞吐率提升了很多！

六、实验结论

1. **SM3 算法结构清晰**，与 SHA-256 相似但更复杂，使用国产 SM3 曲线参数
2. **手写实现易于理解其底层设计**
3. **通过编译器优化和数据结构设计**，性能显著提升至 200+ MB/s